Decision Trees

Sources:

Machine Learning, Tom Mitchell, McGraw-Hill, 1997.

Machine Learning course, Paul G. Allen School

Lecture Notes of David Sontag

Decision Forests, MSR Tech. Rep, A.Criminisi et.al.

Outline

- Decision Trees
- Tree construction
- Overfitting
- Pruning
- Real-valued inputs

Machine Learning in the ER

Physician documentation

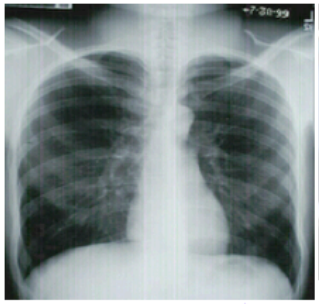
Triage Information (Free text)

MD comments (free text)

Specialist consults











30 min

2 hrs

T=0



Repeated vital signs (continuous values) Measured every 30 s

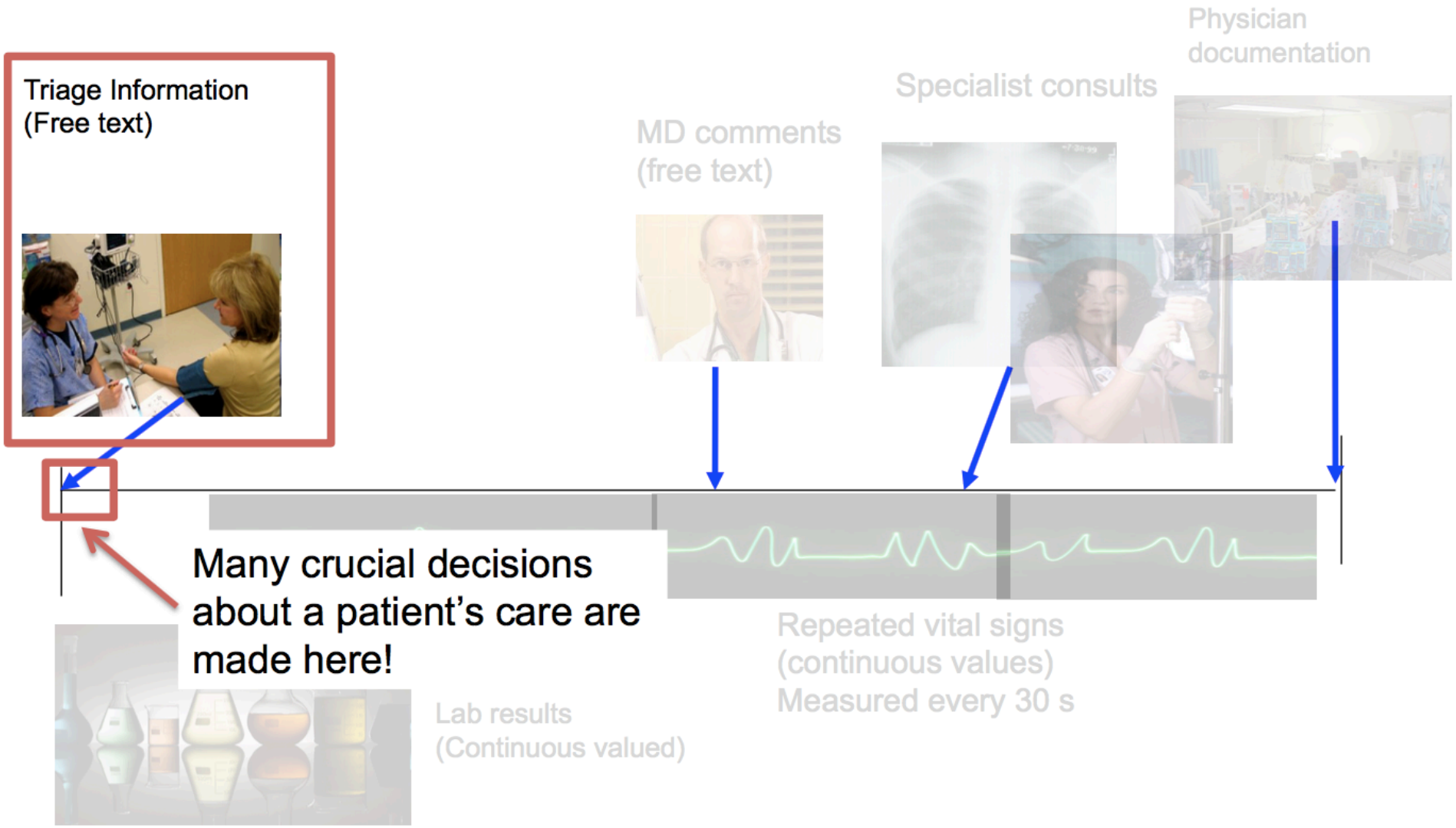
Disposition



Lab results (Continuous valued)

slide by David Sontag

Can we predict infection?

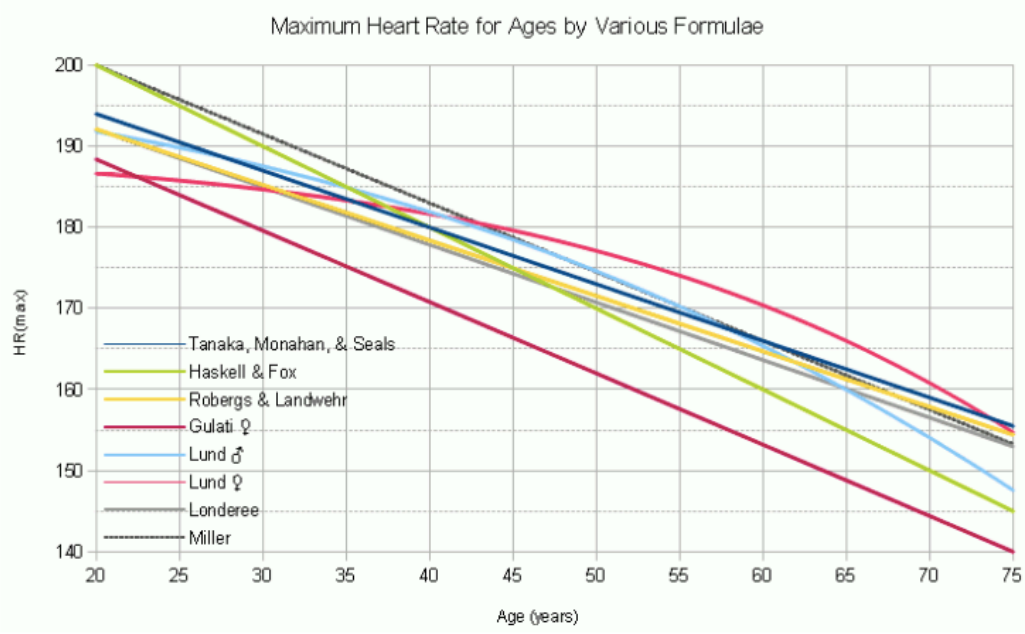


slide by David Sontag

Can we predict infection

- Previous automatic approaches based on simple criteria:
  - Temperature < 96.8 °F or > 100.4 °F
  - Heart rate > 90 beats/min
  - Respiratory rate > 20 breaths/min

- Too simplified... e.g., heart rate depends on age!



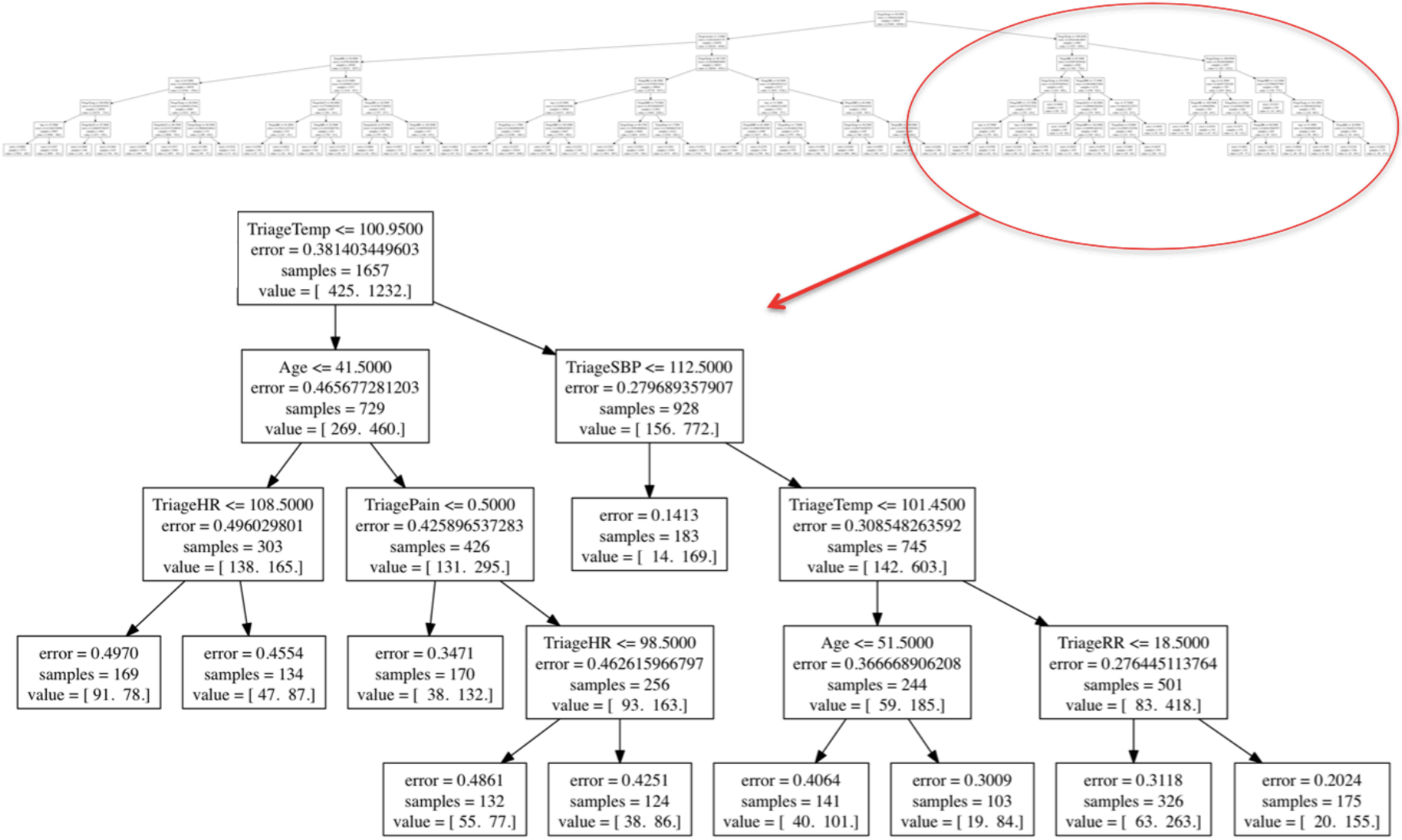
slide by David Sontag

Can we predict infection?

- These are the attributes we have for each patient:
  - Temperature
  - Heart rate (HR)
  - Respiratory rate (RR)
  - Age
  - Acuity and pain level
  - Diastolic and systolic blood pressure (DBP, SBP)
  - Oxygen Saturation (SaO2)
- We have these attributes + label (infection) for 200,000 patients!
- Let's **learn** to classify infection

slide by David Sontag

Predicting infection using decision trees



slide by David Sontag

Example: Image Classification

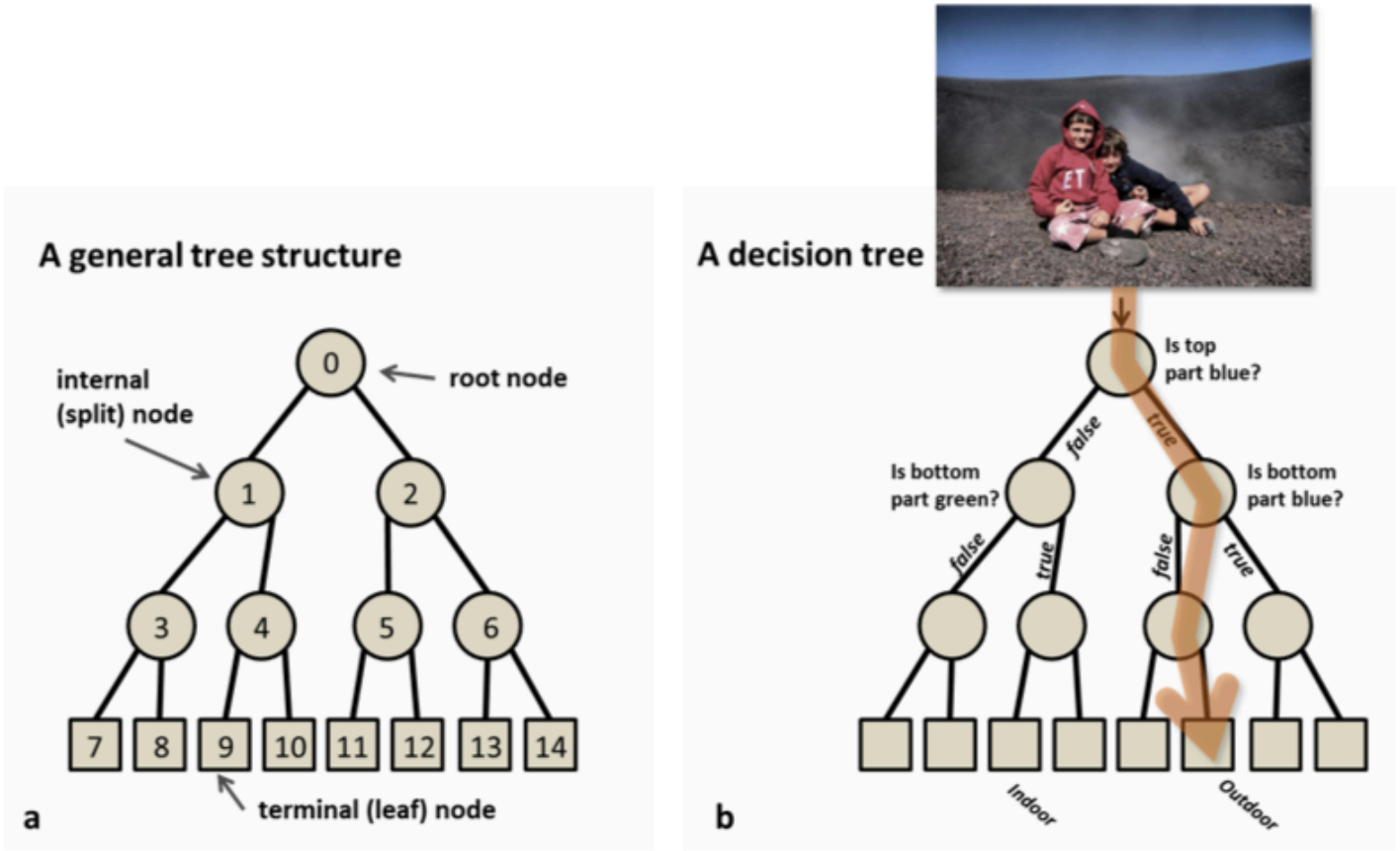
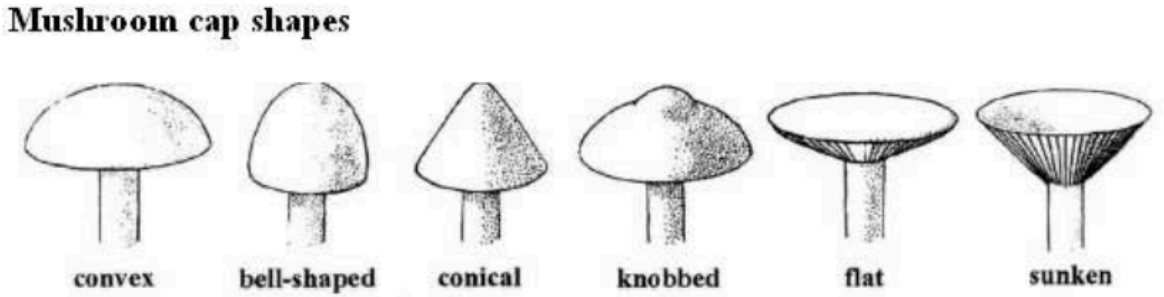


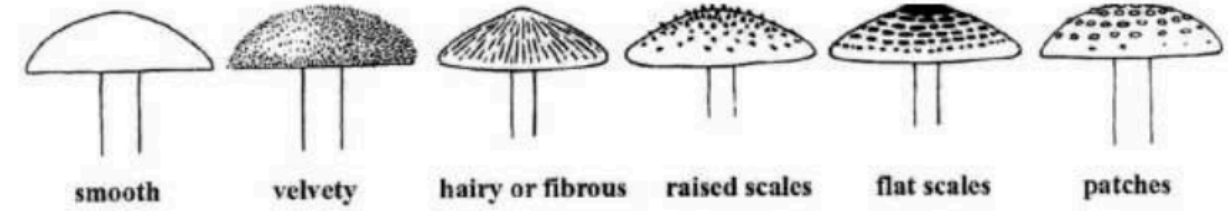
Fig. 2.1: **Decision tree. (a)** A tree is a set of nodes and edges organized in a hierarchical fashion. In contrast to a graph, in a tree there are no loops. Internal nodes are denoted with circles and terminal nodes with squares. **(b)** A decision tree is a tree where each split node stores a test function to be applied to the incoming data. Each leaf stores the final answer (predictor). This figure shows an illustrative decision tree used to figure out whether a photo represents and indoor or outdoor scene.

Example: Mushrooms

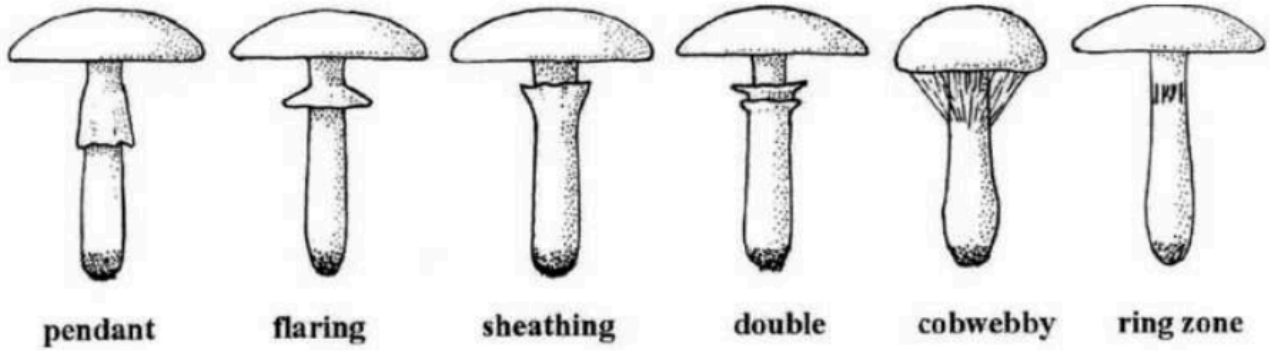




Mushroom cap shapes



Mushroom cap surfaces



Annular rings

http://www.usask.ca/biology/fungi/

slide by Jerry Zhu

Mushroom features

1. **cap-shape:** bell=b, conical=c, convex=x, flat=f, knobbed=k, sunken=s
2. **cap-surface:** fibrous=f, grooves=g, scaly=y, smooth=s
3. **cap-color:** brown=n, buff=b, cinnamon=c, gray=g, green=r, pink=p,purple=u, red=e, white=w, yellow=y
4. **bruises?:** bruises=t,no=f
5. **odor:** almond=a, anise=l, creosote=c, fishy=y, foul=f, musty=m, none=n, pungent=p, spicy=s
6. **gill-attachment:** attached=a, descending=d, free=f, notched=n
7. ...

slide by Jerry Zhu

Two mushrooms

x_1=x,s,n,t,p,f,c,n,k,e,e,s,s,w,w,p,w,o,p,k,s,u

y_1=p

x_2=x,s,y,t,a,f,c,b,k,e,c,s,s,w,w,p,w,o,p,n,n,g

y_2=e

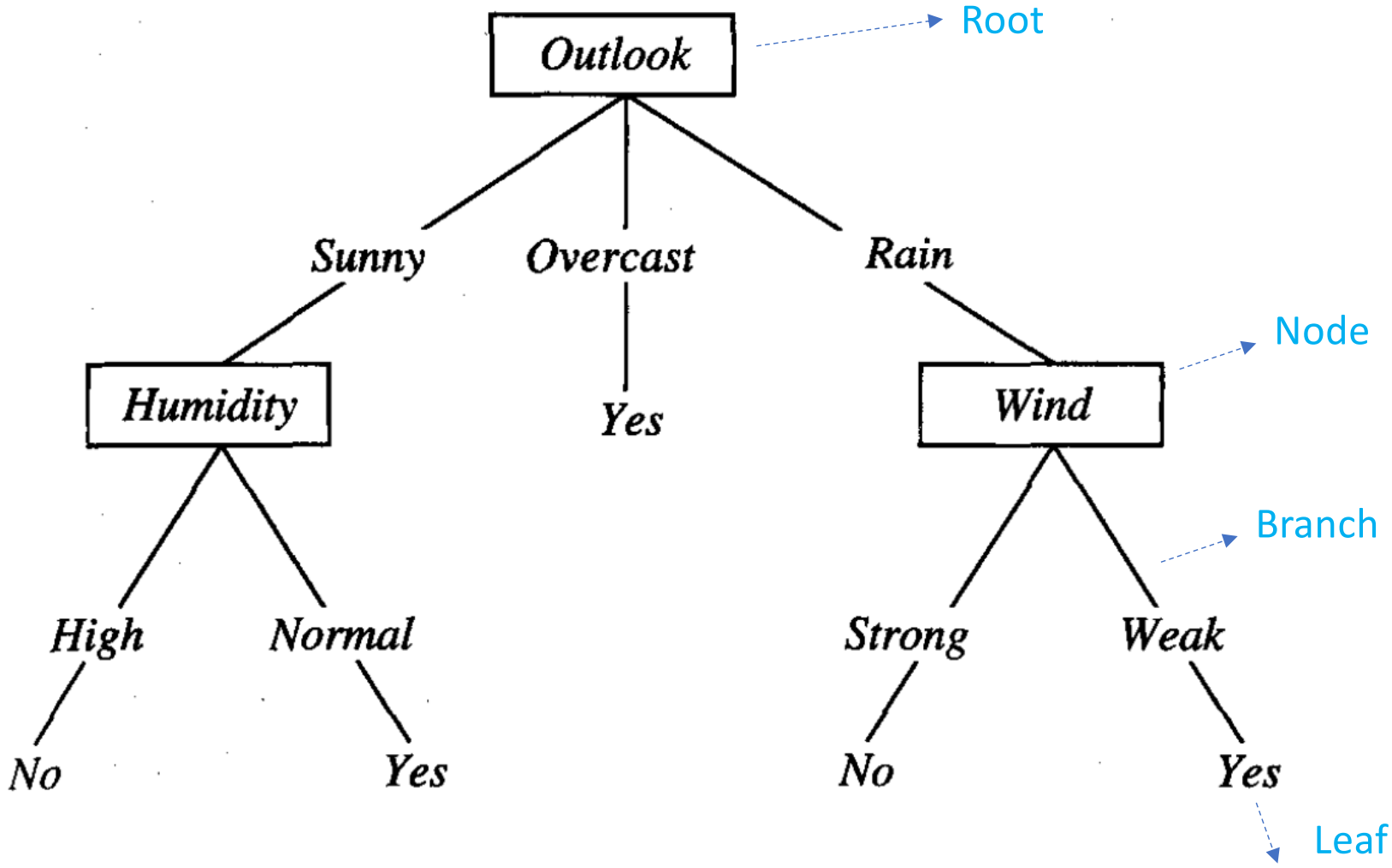
1. cap-shape: bell=b,conical=c,convex=x,flat=f, knobbed=k,sunken=s
2. cap-surface: fibrous=f,grooves=g,scaly=y,smooth=s
3. cap-color: brown=n,buff=b,cinnamon=c,gray=g,green=r, pink=p,purple=u,red=e,white=w,yellow=y
4. …

slide by Jerry Zhu

Decision Trees

- One of the most widely used and practical methods for inductive inference.
- Decision tree is a learning method for approximating discrete-valued target functions.
- Learned trees can also be re-represented as sets of if-then rules to improve human readability.

Decision Tree Representation



Classifies Saturday mornings according to whether they are suitable for tennis.

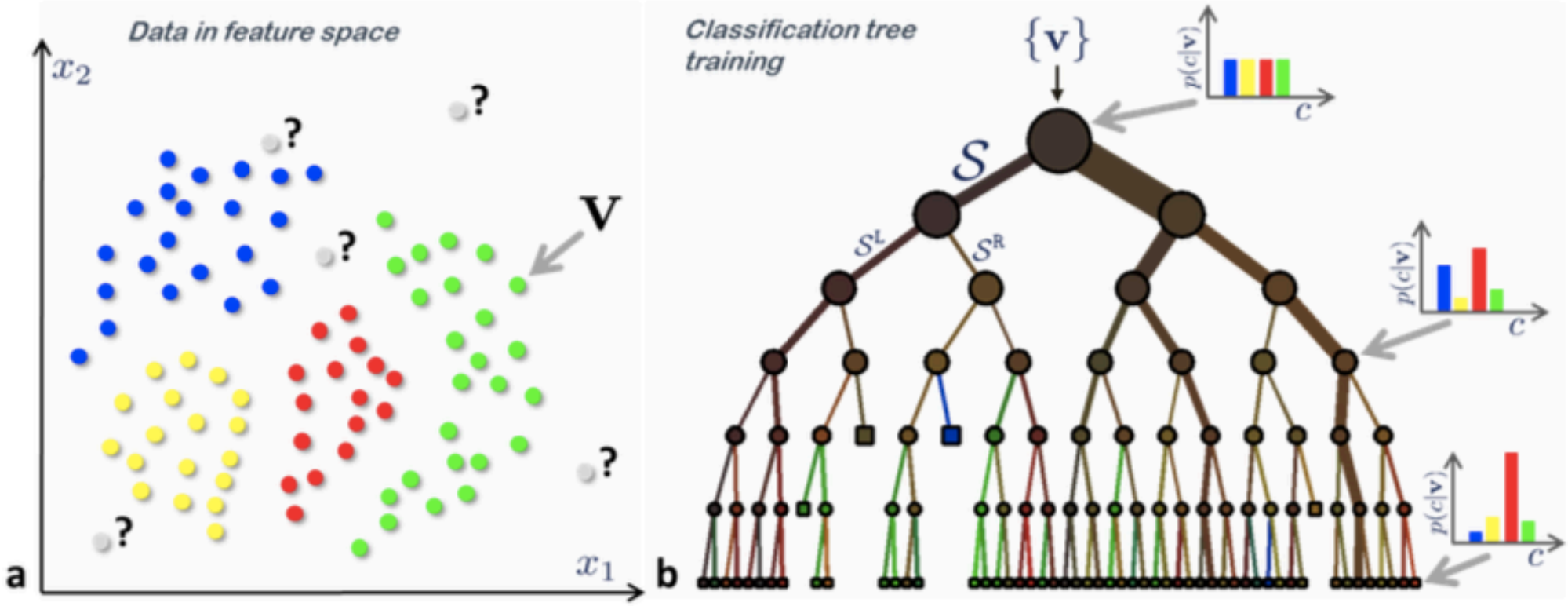


Fig. 3.1: **Classification: training data and tree training. (a)** Input data points. The ground-truth label of training points is denoted with different colours. Grey circles indicate unlabelled, previously unseen test data. **(b)** A binary classification tree. During training a set of labelled training points $\{\mathbf{v}\}$ is used to optimize the parameters of the tree. In a classification tree the entropy of the class distributions associated with different nodes decreases (the confidence increases) when going from the root towards the leaves.

Decision Tree Representation

- Classify instances by sorting them down the tree from the root to some leaf node.
- Leaf nodes provide the classification of the instance.
- Each node in the tree specifies a test of some *attribute* of the instance.
- Each branch descending from that node corresponds to one of the possible values for this attribute.
- An instance is classified by
  - Starting at the root node of the tree, testing the attribute specified for this node, then moving down the tree branch corresponding to the value of the attribute in the given example.
  - Repeat this process for the subtree rooted at the new node.

Decision Tree Hypothesis Space

- **Internal nodes** test the value of particular features x_j and branch according to the results of the test.
- **Leaf nodes** specify the class $h(\mathbf{x})$.



Suppose the features are **Outlook** (x_1), **Temperature** (x_2), **Humidity** (x_3), and **Wind** (x_4). Then the feature vector $\mathbf{x} = (Sunny, Hot, High, Strong)$ will be classified as **No**. The **Temperature** feature is irrelevant.

Decision Trees

- Each path from the root to a leaf corresponds to a sequence of attribute tests.
- Tree itself is a combination of tests.
- The decision tree in the previous slide corresponds to the expression:

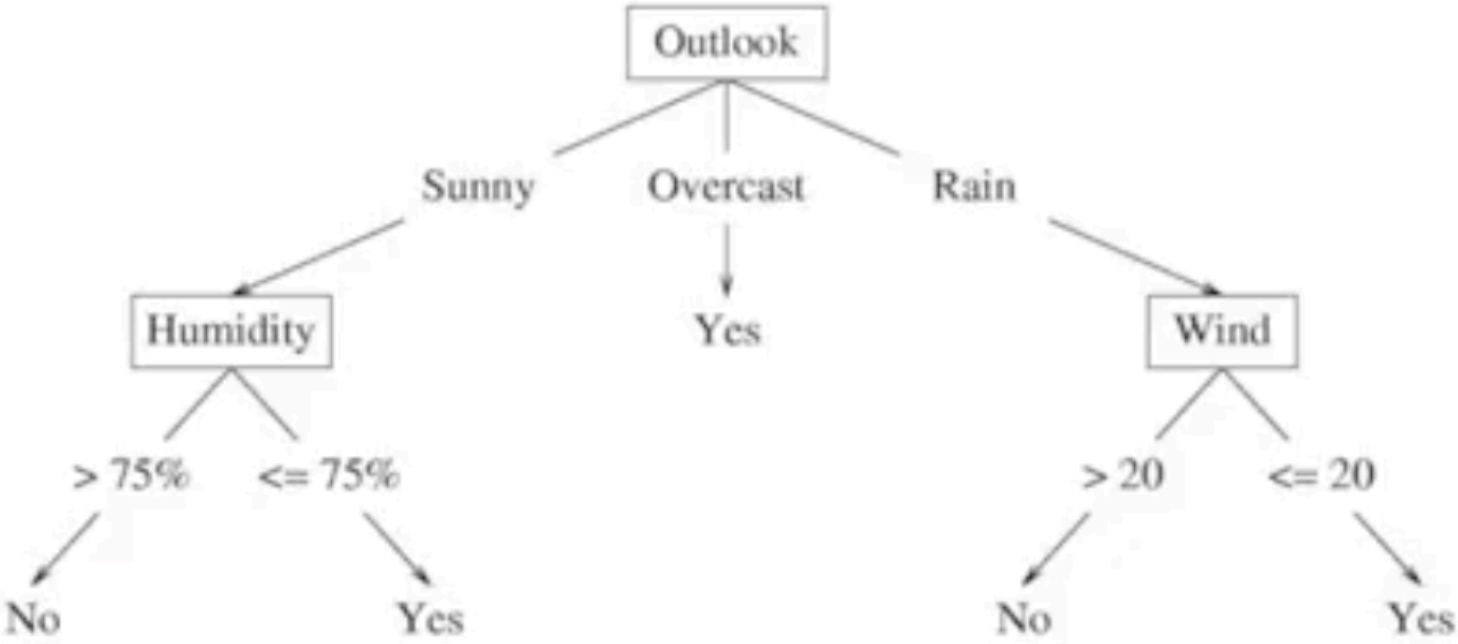
$$(Outlook = Sunny \wedge Humidity = Normal)$$
$$\vee \quad (Outlook = Overcast)$$
$$\vee \quad (Outlook = Rain \wedge Wind = Weak)$$

Appropriate Problems for Decision Tree Learning

- Decision tree learning is generally best suited for the following problems:
  - **Instances are represented by attribute-value pairs**. Instances are described by a fixed set of attributes (e.g. Temperature) and their values (e.g. Hot). It is best if the attribute takes on a small number of possible values (e.g. Hot, Mild, Cold). Extensions allow real-valued attributes (e.g. Temperature is represented numerically.
  - **The target function has discrete output values**. (e.g. Yes or No in the PlayTennis example).
  - **The training data may contain errors**: Decision trees are robust to errors.
  - **The training data may contain missing attribute values:** (e.g. Humidity of the day is known for only some of the training examples).

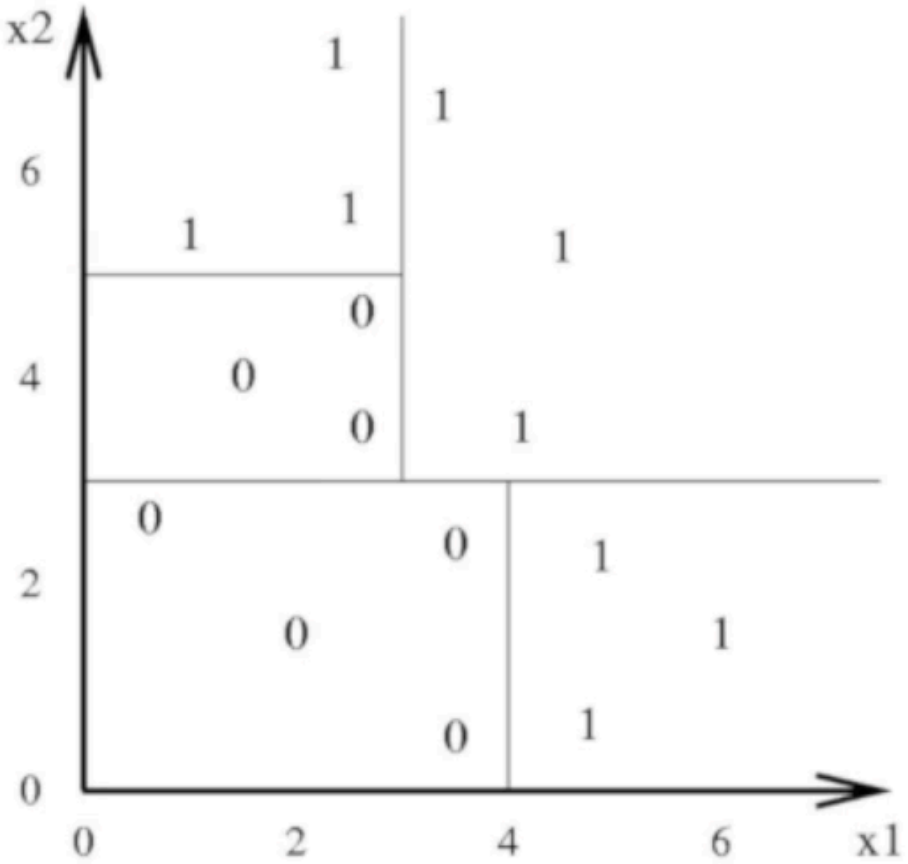
Decision Tree Hypothesis Space

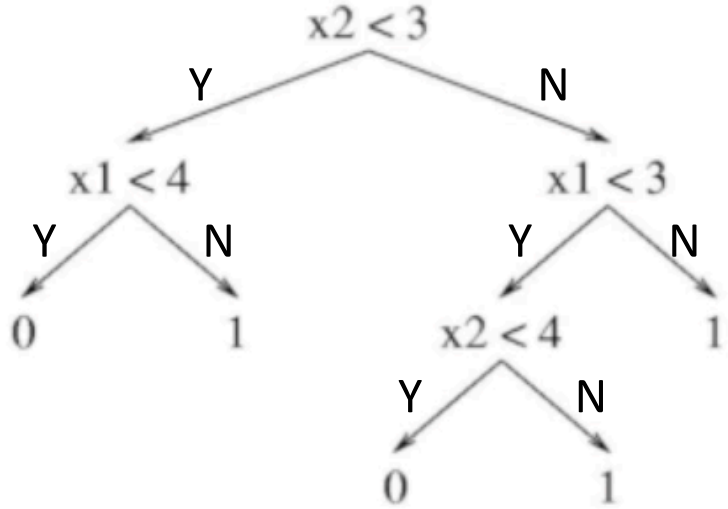
If the features are continuous, internal nodes may test the value of a feature against a threshold.



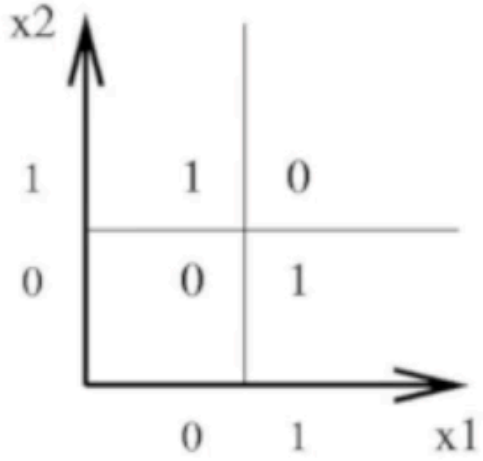
Decision Tree Decision Boundaries

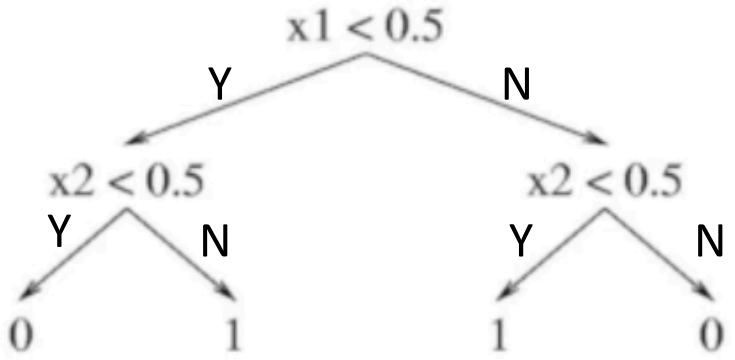
Decision trees divide the feature space into axis-parallel rectangles, and label each rectangle with one of the K classes.





Decision Trees Can Represent Any Boolean Function

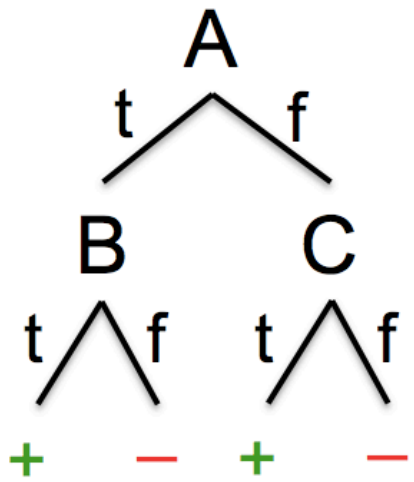


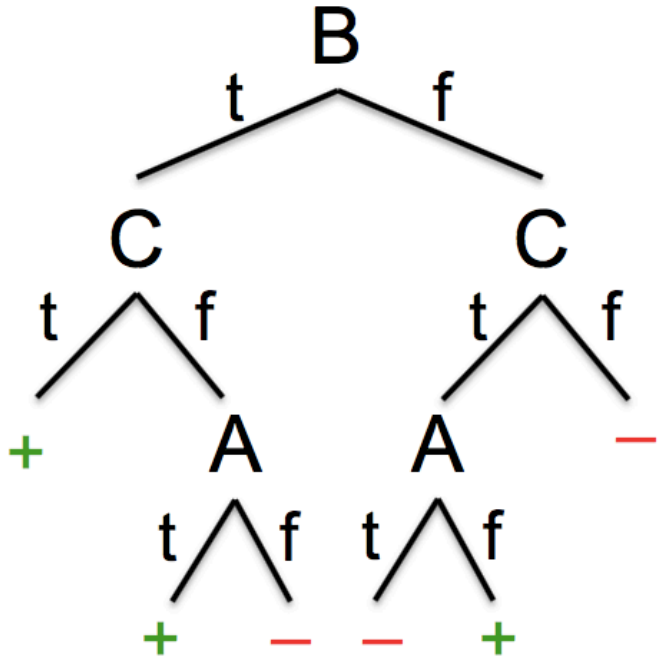


The tree will in the worst case require exponentially many nodes, however.

Are all decision trees equal?

- Many trees can represent the same concept
- But, not all trees will have the same size
  - e.g., $\phi = (A \wedge B) \vee (\neg A \wedge C)$ — ((A and B) or (not A and C))





- Which tree do we prefer?

slide by David Sontag

Decision Trees Provide Variable-Size Hypothesis Space

As the number of nodes (or depth) of tree increases, the hypothesis space grows

- **depth 1** ("decision stump") can represent any boolean function of one feature.
- **depth 2** Any boolean function of two features; some boolean functions involving three features (e.g., $(x_1 \wedge x_2) \vee (\neg x_1 \wedge \neg x_3)$
- **etc.**

Applications of Decision Trees

- Applied to the classification of
  - Medical patients by their disease
  - Equipment malfunctions by their cause
  - Loan applicants by their likelihood of defaulting on payments

Basic Decision Tree Learning Algorithm

- Basic algorithm: ID3 (Quinlan 1986)
- Successor: C4.5 (Quinlan 1993)
- Beginning question:
  - Which attribute should be tested at the root of the tree?
- Answer:
  - Evaluate each instance attribute using a statistical test to determine how well it alone classifies the training samples.
  - Select the best attribute and use it at the root node of the tree.
  - Repeat the process using the training examples associated with each descendant node.

Learning Algorithm for Decision Trees

The same basic learning algorithm has been discovered by many people independently:

GROWTREE(S)
if ($y = 0$ for all $\langle \mathbf{x}, y \rangle \in S$) **return** new leaf(0)
else if ($y = 1$ for all $\langle \mathbf{x}, y \rangle \in S$) **return** new leaf(1)
else
    choose best attribute x_j
    $S_0 =$ all $\langle \mathbf{x}, y \rangle \in S$ with $x_j = 0$;
    $S_1 =$ all $\langle \mathbf{x}, y \rangle \in S$ with $x_j = 1$;
    return new node(x_j, GROWTREE(S_0), GROWTREE(S_1))

S is a collection of instances, containing positive and negative examples of target.

Which attribute is the best classifier?

- We would like to choose the attribute that is most useful for classifying examples.
  - Good split if we are more certain about the classification after the split.
- A good quantitative measure of the worth of the attribute:
  - **Information gain**: measures how well a given attribute separates the training examples according to their target classification.

Information (surprise)

Let V be a random variable with the following probability distribution:

$P(V=0)$	$P(V=1)$
0.2	0.8

The *surprise*, $S(V=v)$ of each value of V is defined to be

$$S(V=v) = -\lg P(V=v).$$

An event with probability 1 gives us zero surprise.

An event with probability 0 gives us infinite surprise!

It turns out that the surprise is equal to the number of bits of information that need to be transmitted to a recipient who knows the probabilities of the results.

This is also called the *description length* of $V=v$.

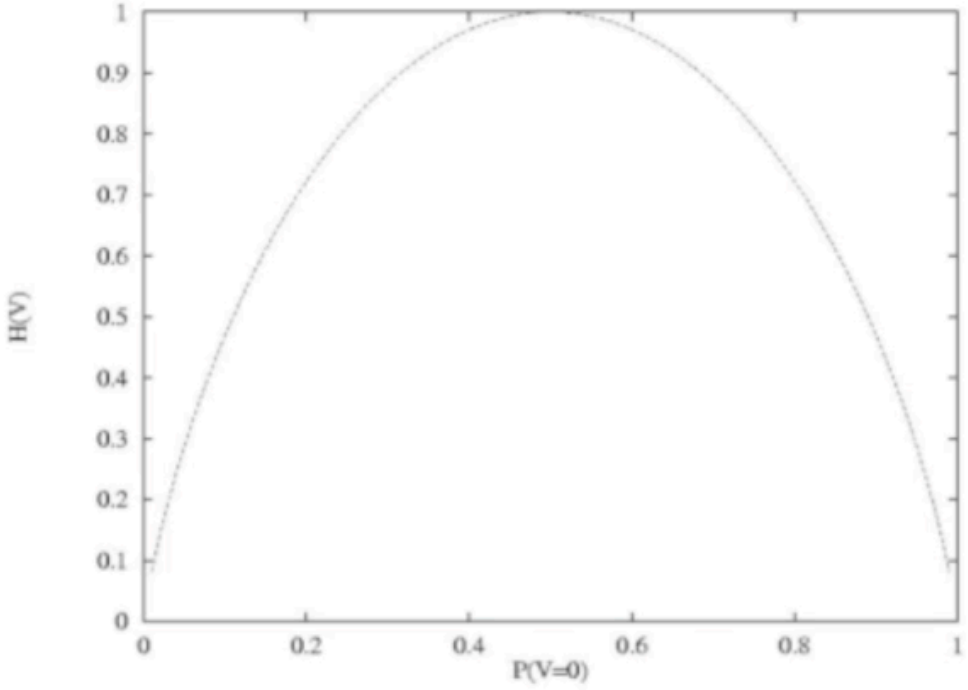
Fractional bits only make sense if they are part of a longer message (e.g., describe a whole sequence of coin tosses).

Entropy

The *entropy* of V, denoted $H(V)$ is defined as follows:

$$H(V) = \sum_{v=0}^{1} -P(H = v) \lg P(H = v).$$

This is the average surprise of describing the result of one "trial" of V (one coin toss).



Entropy can be viewed as a measure of uncertainty.

Which attribute is the best classifier?

- Measure of effectiveness of an attribute in classifying training data: **Information gain**
- Expected reduction in entropy caused by partitioning the examples according to this attribute.
- The information gain Gain(S,A) of an attribute A, relative to a collection of samples S is:

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- Values(A): set of all possible values for attribute A.
- S_v : subset of S for which attribute A has value v:

$$S_v = \{s \in S | A(s) = v\}$$

Information Gain

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- First term: entropy of the original collection S.
- Second term: expected value of entropy after S has been partitioned using attribute A.
- Gain:
  - Expected reduction in entropy caused by knowing the value of the attribute A.
  - The information provided about the target function value, given the value of attribute A.

Information Gain

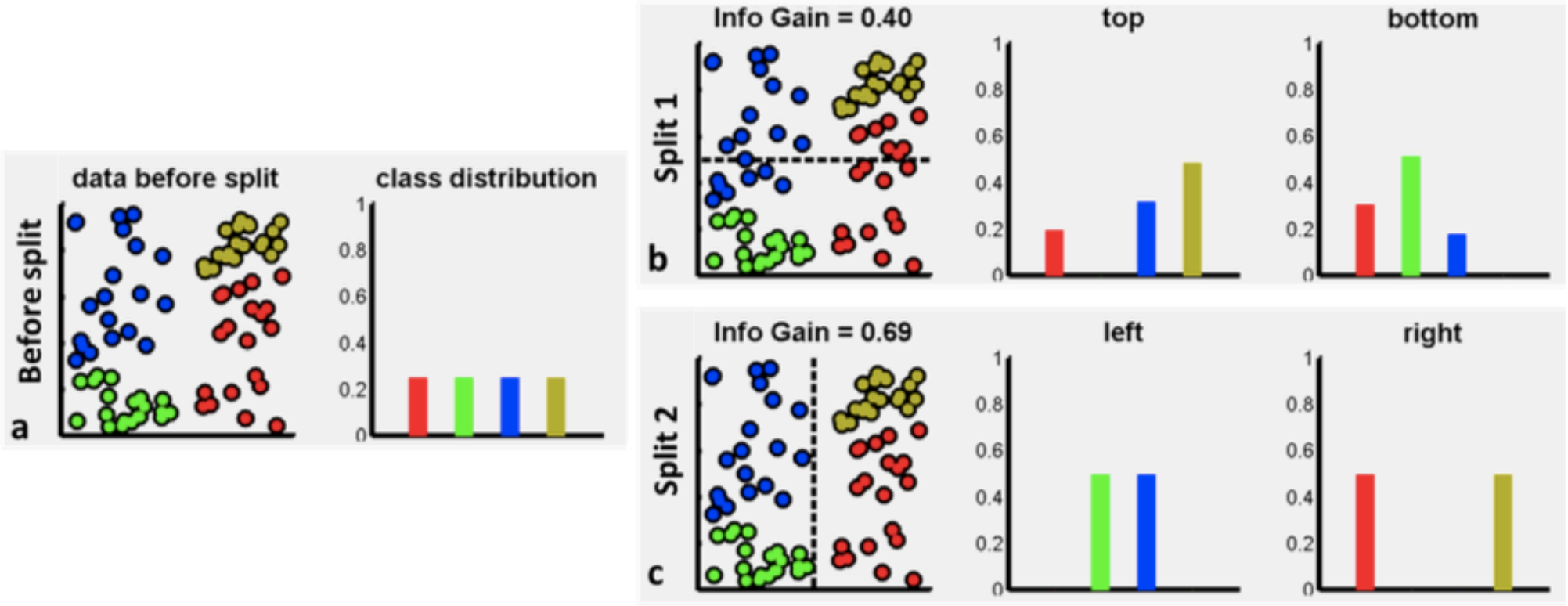


Fig. 2.3: **Information gain for discrete, non-parametric distributions.** **(a)** Dataset $\mathcal{S}$ before a split. **(b)** After a horizontal split. **(c)** After a vertical split.

Example

Day	*Outlook*	*Temperature*	*Humidity*	*Wind*	*PlayTennis*
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

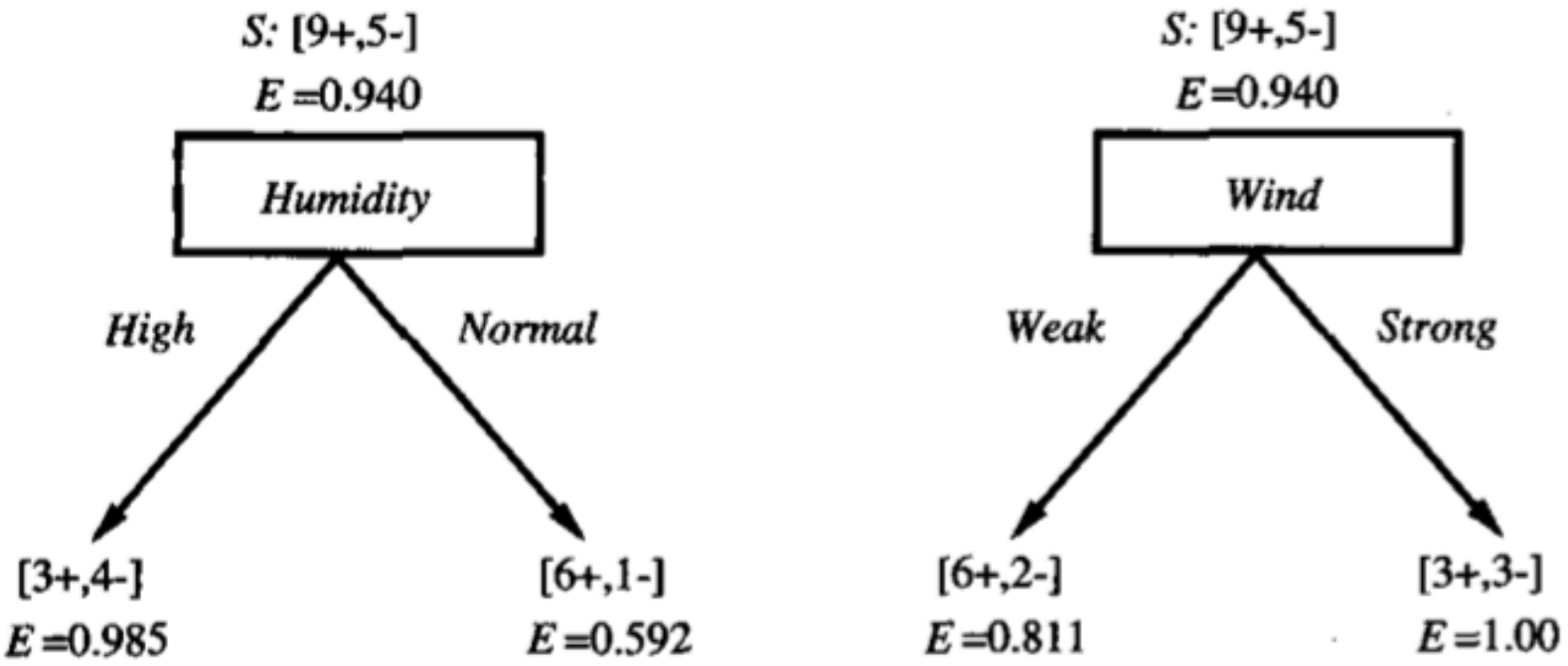
TABLE 3.2
Training examples for the target concept *PlayTennis*.

Example

- S: collection of play tennis days, 14 samples, [9+,5-]
- Attribute: **Wind**, which can be *Weak* or *Strong*.

$$
\begin{aligned}
Values(Wind) &= Weak, Strong \\
S &= [9+, 5-] \\
S_{Weak} &\leftarrow [6+, 2-] \\
S_{Strong} &\leftarrow [3+, 3-] \\
Gain(S, Wind) &= Entropy(S) - \sum_{v \in \{Weak, Strong\}} \frac{|S_v|}{|S|} Entropy(S_v) \\
&= Entropy(S) - (8/14) Entropy(S_{Weak}) \\
&\quad - (6/14) Entropy(S_{Strong}) \\
&= 0.940 - (8/14)0.811 - (6/14)1.00 \\
&= 0.048
\end{aligned}
$$

Which attribute is the best classifier?



Gain (S, Humidity)

= .940 - (7/14).985 - (7/14).592
= .151

Gain (S, Wind)

= .940 - (8/14).811 - (6/14)1.0
= .048

Humidity provides greater information gain than *Wind*, relative to the target classification. Here, E stands for entropy and S for the original collection of examples. Given an initial collection S of 9 positive and 5 negative examples, [9+, 5−], sorting these by their *Humidity* produces collections of [3+, 4−] (*Humidity* = *High*) and [6+, 1−] (*Humidity* = *Normal*). The information gained by this partitioning is .151, compared to a gain of only .048 for the attribute *Wind*.

Example

Day	*Outlook*	*Temperature*	*Humidity*	*Wind*	*PlayTennis*
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

TABLE 3.2
Training examples for the target concept *PlayTennis*.

Example

- Which attribute should be tested first in the tree?
- ID3 (Iterative Dichotomiser 3): determine the information gain for each attribute (Outlook, Temperature, Humidity, Wind), then select the one with the highest information gain.

$$Gain(S, Outlook) = 0.246$$ → Highest

$$Gain(S, Humidity) = 0.151$$

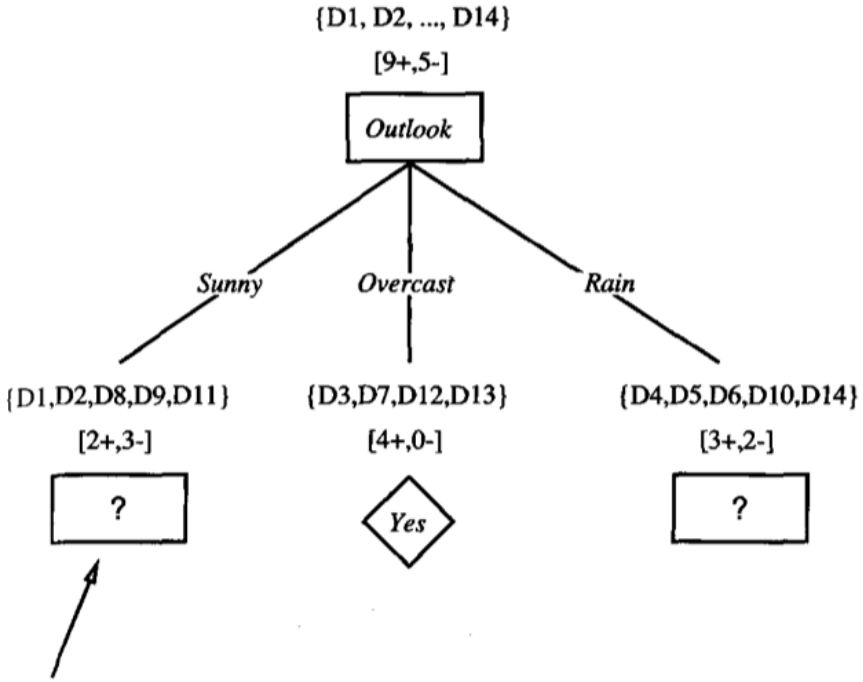
$$Gain(S, Wind) = 0.048$$

$$Gain(S, Temperature) = 0.029$$

Example

- Outlook attribute provides the best prediction of the target attribute, since its information gain is highest.
  - Outlook is selected as the decision attribute at the root node and branches are created below the root for each of its possible values.
  - The process of selecting a new attribute and partitioning the training examples is repeated for each nonterminal descendant node, using only the training examples associated with that node.
  - Attributes that have been incorporated higher in the tree are excluded, so that any attribute can appear at most once along any path through the tree.

Example: Partially Learned Tree



S_{sunny} = {D1,D2,D8,D9,D11}

Gain (S_{sunny}, *Humidity*) = .970 - (3/5) 0.0 - (2/5) 0.0 = .970

Gain (S_{sunny}, *Temperature*) = .970 - (2/5) 0.0 - (2/5) 1.0 - (1/5) 0.0 = .570

Gain (S_{sunny}, *Wind*) = .970 - (2/5) 1.0 - (3/5) .918 = .019

The partially learned decision tree resulting from the first step of ID3. The training examples are sorted to the corresponding descendant nodes. The *Overcast* descendant has only positive examples and therefore becomes a leaf node with classification *Yes*. The other two nodes will be further expanded, by selecting the attribute with highest information gain relative to the new subsets of examples.

When to stop?

- This process continues until either of the two conditions are met:
  - Every attribute has already been included along this path through the tree.
  - The training examples associated with this leaf node all have the same target attribute value (their entropy is zero).

Hypothesis Space Search

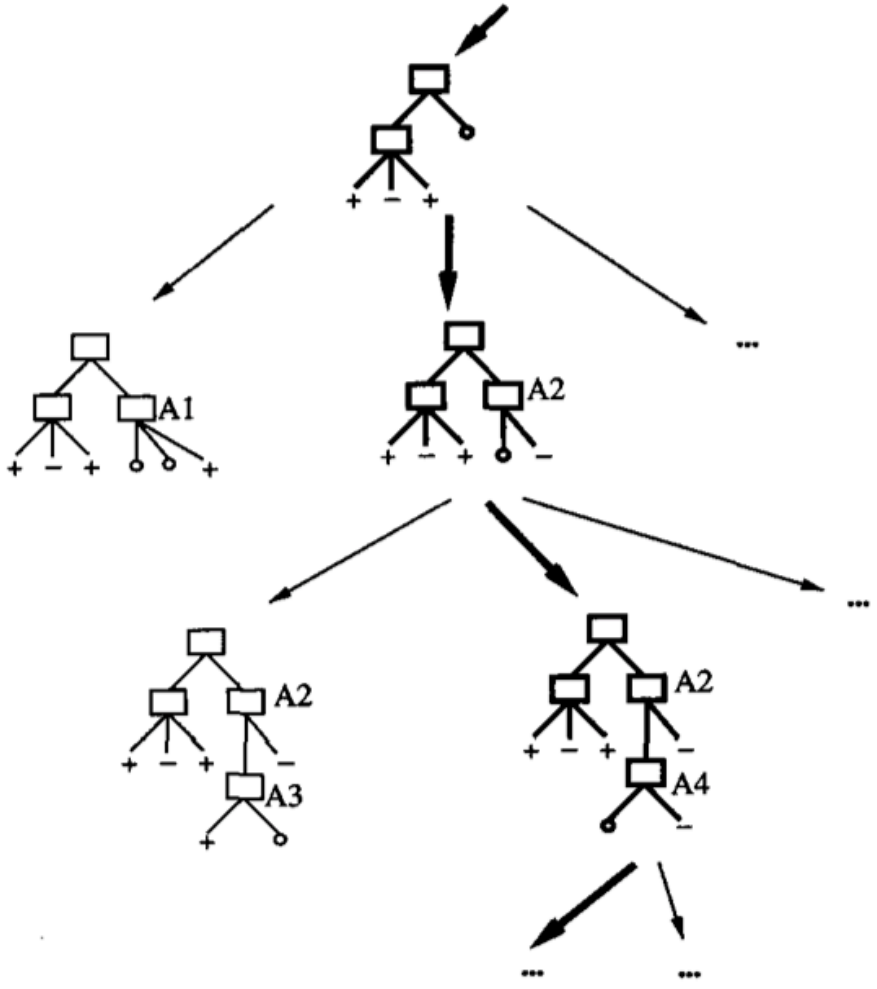


FIGURE 3.5
Hypothesis space search by ID3. ID3 searches through the space of possible decision trees from simplest to increasingly complex, guided by the *information gain heuristic.*

Hypothesis Space Search

- ID3 in its pure form performs no backtracking:
  - Once it selects an attribute to test at a particular level in the tree, it never backtracks to reconsider this choice.
  - Hence, it is susceptible to the risk of converging to a local minimum.
- ID3 search strategy:
  - Selects in favor of shorter trees over longer ones
  - Selects trees that place the attributes with highest information gain closest to the root.

ID3($Examples$, $Target_attribute$, $Attributes$)

Examples are the training examples. Target_attribute is the attribute whose value is to be predicted by the tree. Attributes is a list of other attributes that may be tested by the learned decision tree. Returns a decision tree that correctly classifies the given Examples.

- Create a *Root* node for the tree
- If all $Examples$ are positive, Return the single-node tree *Root*, with label = +
- If all $Examples$ are negative, Return the single-node tree *Root*, with label = −
- If $Attributes$ is empty, Return the single-node tree *Root*, with label = most common value of $Target_attribute$ in $Examples$
- Otherwise Begin
  - $A \leftarrow$ the attribute from $Attributes$ that best* classifies $Examples$
  - The decision attribute for $Root \leftarrow A$
  - For each possible value, v_i, of A,
    - Add a new tree branch below *Root*, corresponding to the test $A = v_i$
    - Let $Examples_{v_i}$ be the subset of $Examples$ that have value v_i for A
    - If $Examples_{v_i}$ is empty
      - Then below this new branch add a leaf node with label = most common value of $Target_attribute$ in $Examples$
      - Else below this new branch add the subtree
        ID3($Examples_{v_i}$, $Target_attribute$, $Attributes - \{A\}$))
- End
- Return *Root*

* The best attribute is the one with highest *information gain*, as defined in Equation (3.4).

TABLE 3.1

Summary of the ID3 algorithm specialized to learning boolean-valued functions. ID3 is a greedy algorithm that grows the tree top-down, at each node selecting the attribute that best classifies the local training examples. This process continues until the tree perfectly classifies the training examples, or until all attributes have been used.

Avoiding Overfitting the Data

- The ID3 algorithm grows each branch of the tree to perfectly classify the training samples.
- This can lead to *overfitting* if:
  - There is noise in the data or
  - When the number of training examples is too small to produce a representative sample of the true target function.
- A hypothesis overfits the training examples if some other hypothesis that fits the training examples less well actually performs better over the test set.

Overfitting

Definition: Given a hypothesis space H, a hypothesis $h \in H$ is said to **overfit** the training data if there exists some alternative hypothesis $h' \in H$, such that h has smaller error than h' over the training examples, but h' has a smaller error than h over the entire distribution of instances.

Overfitting in Decision Trees

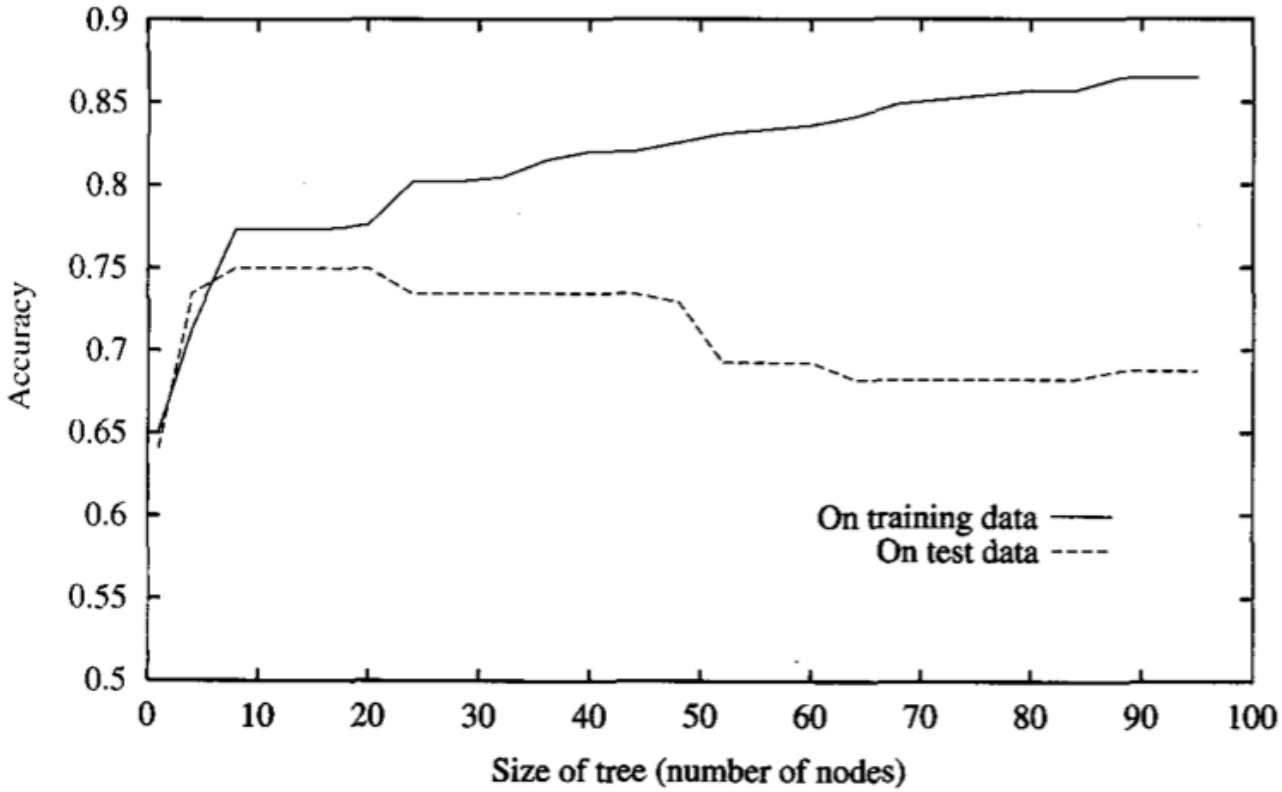


FIGURE 3.6
Overfitting in decision tree learning. As ID3 adds new nodes to grow the decision tree, the accuracy of the tree measured over the training examples increases monotonically. However, when measured over a set of test examples independent of the training examples, accuracy first increases, then decreases. Software and data for experimenting with variations on this plot are available on the World Wide Web at http://www.cs.cmu.edu/~tom/mlbook.html.

Overfitting

- How can it be possible for tree h to fit the training examples better than h', but for it to perform more poorly over subsequent samples?
  - This can happen if training samples contain random errors or noise.
  - Consider adding the following positive training example, incorrectly labeled as negative:

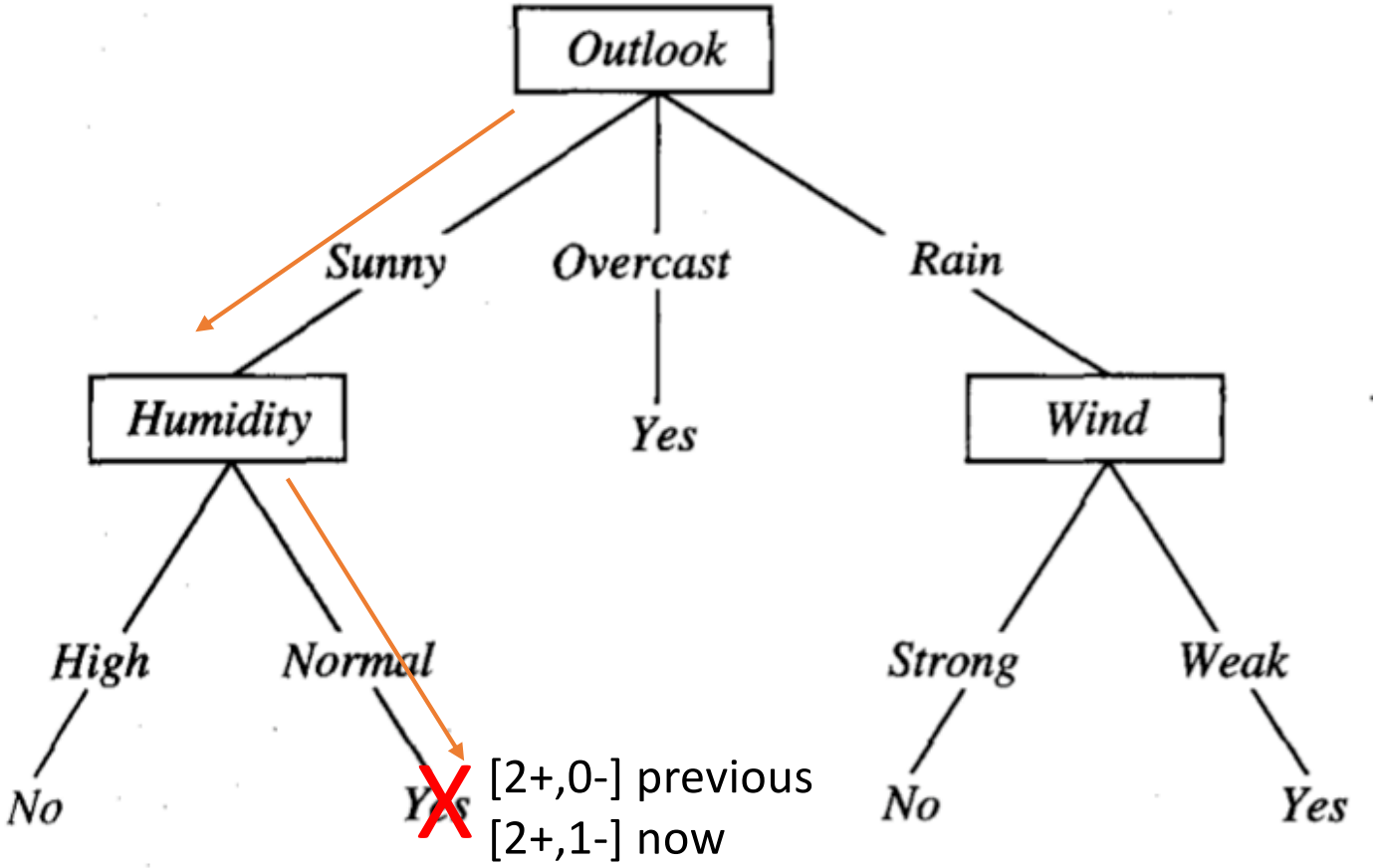
$$\langle Outlook = Sunny, Temperature = Hot, Humidity = Normal,$$
$$Wind = Strong, PlayTennis = No \rangle$$

  - Addition of this incorrect example will cause ID3 to construct a more complex tree.

Overfitting

$\langle Outlook = Sunny, Temperature = Hot, Humidity = Normal,$

$Wind = Strong, PlayTennis = No \rangle$



[2+,0-] previous
[2+,1-] now

Will include another split --> Decision tree will be more complex, overfit

Overfitting

- Overfitting can happen when the training data is noise-free, especially when small number of examples are associated with leaf nodes.
  - Coincidental regularities may occur
  - Some attributes happen to partition the examples very well, despite being unrelated to the actual target function.

Approaches to Avoid Overfitting

1. Stop growing the tree earlier, before it reaches the point where it perfectly classifies the training data. (Difficult to estimate precisely when to stop)
2. Allow the tree to overfit the data, and then post-prune the tree. This approach has been found to be more successful in practice.

Reduced Error Pruning

- Consider each decision node for pruning
- Pruning means:
  - Remove the subtree rooted at that node, making it a leaf node, and assign it the most common classification of the training examples affiliated with that node.
- Nodes are removed only if the resulting pruned tree performs no worse than the original over the validation set.
- Pruning continues until pruning becomes harmful.

Reduced-Error Pruning

Split data into *training* and *validation* set

Do until further pruning is harmful:

1. Evaluate impact on *validation* set of pruning each possible node (plus those below it)
2. Greedily remove the one that most improves *validation* set accuracy

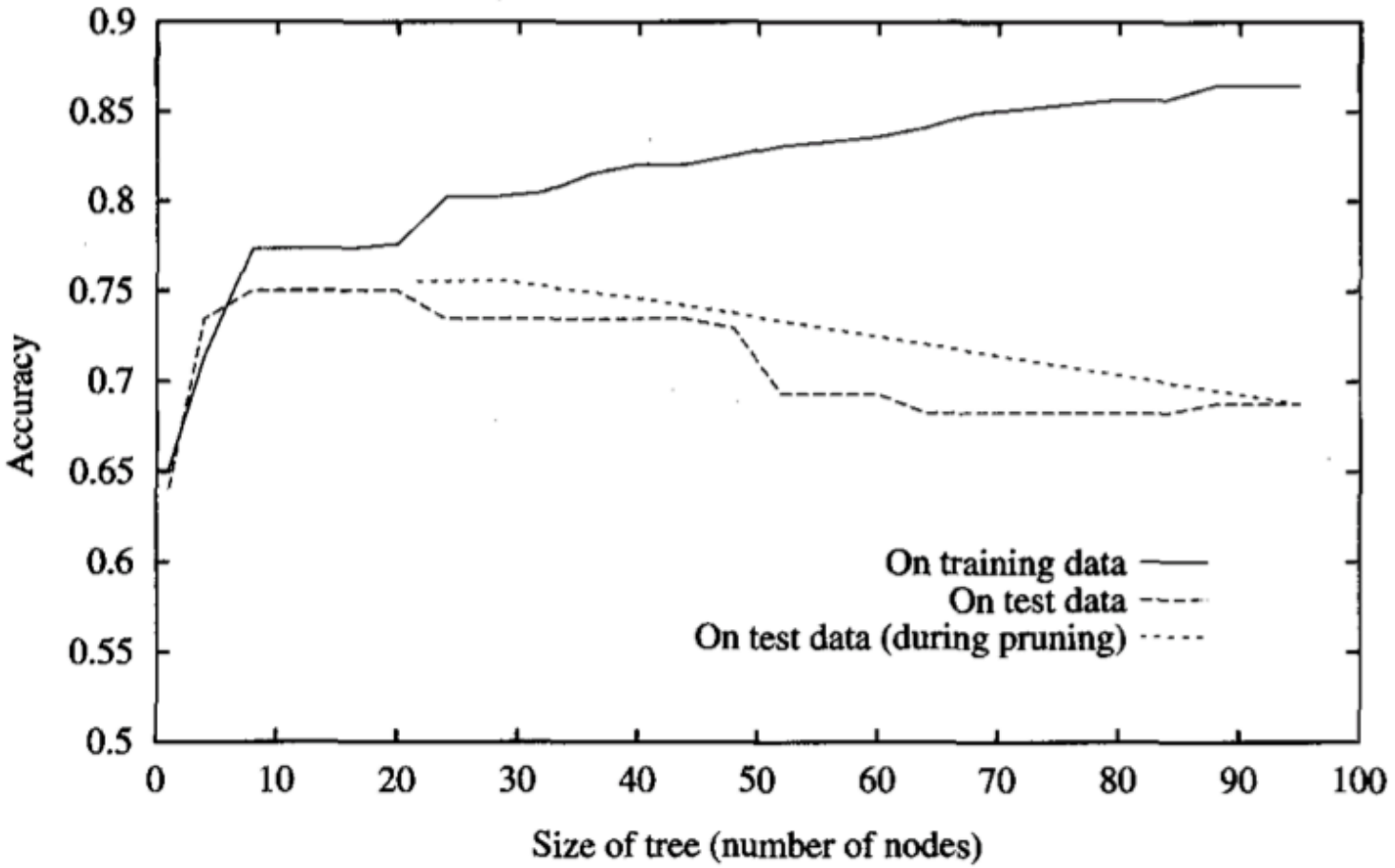


FIGURE 3.7
Effect of reduced-error pruning in decision tree learning. This plot shows the same curves of training and test set accuracy as in Figure 3.6. In addition, it shows the impact of reduced error pruning of the tree produced by ID3. Notice the increase in accuracy over the test set as nodes are pruned from the tree. Here, the validation set used for pruning is distinct from both the training and test sets.

Rule Post-Pruning

Rule post-pruning involves the following steps:

1. Infer the decision tree from the training set, growing the tree until the training data is fit as well as possible and allowing overfitting to occur.
2. Convert the learned tree into an equivalent set of rules by creating one rule for each path from the root node to a leaf node.
3. Prune (generalize) each rule by removing any preconditions that result in improving its estimated accuracy.
4. Sort the pruned rules by their estimated accuracy, and consider them in this sequence when classifying subsequent instances.

Rule Post-Pruning

For example, the leftmost path of the tree in Figure 3.1 is translated into the rule

IF $(Outlook = Sunny) \wedge (Humidity = High)$

THEN $PlayTennis = No$

Next, each such rule is pruned by removing any antecedent, or precondition, whose removal does not worsen its estimated accuracy. Given the above rule, for example, rule post-pruning would consider removing the preconditions $(Outlook = Sunny)$ and $(Humidity = High)$. It would select whichever of these pruning steps produced the greatest improvement in estimated rule accuracy, then consider pruning the second precondition as a further pruning step. No pruning step is performed if it reduces the estimated rule accuracy.

Continous-Valued Attributes

- Continous-valued decision attributes can be incorporated into the learned tree.
- Define new discrete-valued attributes that partition the continous attribute into a discrete set of intervals.
- If attribute A is continous valued, create a new boolean attribute A_c, which is true if $A < c$ and false otherwise.
- How to select the best value for c?

Example

- Wish to include the continous-valued attribute *Temperature* in the PlayTennis example:

Temperature:	40	48	60	72	80	90
PlayTennis:	No	No	Yes	Yes	Yes	No

- What threshold should be defined on Temperature?
  - Two candidate thesholds:
    - (48+60)/2
    - (80+90)/2
    - Compute the information gain for each and select the best.

Alternative Measures for Selecting Attributes

- There is a natural bias in the information gain measure that favors attributes with many values over those with few values.
- Extreme example:
  - Attibute *Date,* has a large number of possible values.
  - If we add Date to the previous example, it would have the highest information gain. Since date alone perfectly predicts the target attribute (PlayTennis).
  - It would be selected as the decision attribute at the root node and lead to a tree of depth one, which perfectly classifies data.
  - Tree would perform poorly on new examples. Since date is not a useful predictor.

Alternative Measures for Selecting Attributes

- What is wrong with the attribute Date?
  - It has so many possible values that it separates the training examples into very small subsets.
  - Hence, it will have a very high information gain relative to the training examples, despite it is a poor predictor of the target function over unseen instances.
- To avoid this difficulty, select decision attributes based on some measure other than the information gain.

Alternative Measures for Selecting Attributes

- One alternative measure is the **gain ratio**:

$$GainRatio(S, A) \equiv \frac{Gain(S, A)}{SplitInformation(S, A)}$$

- Where

$$SplitInformation(S, A) \equiv -\sum_{i=1}^{c} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

- S_1 through S_c are the c subsets of examples resulting from partitioning S by the c-valued attribute A.

Gain Ratio

- *SplitInformation* is sensitive to how broadly and uniformly the attribute splits the data.
- *SplitInformation* is the entropy of S with respect to the values of the attribute A.
- For example, consider a collection of n examples that are completely seperated by attribute A (e.g. *Date*).
  - The *SplitInformation* will be log_2n.
  - A Boolean attribute B that splits the same n examples in half will have *SplitInformation* of 1.
  - If attributes A and B have the same information gain, B will score higher according to GainRatio measure.

Training Examples with Missing Attribute Values

- Suppose we want to calculate Gain(S,A) at node n to evaluate whether the attribute A is the best attribute to test at this node. Suppose <x,c(x)> is a training example in S and assume A(x) is unknown for that example.
- Several strategies:
  - Assign it the value that is most common among the training examples at node n.
  - Assign it the most common value among examples at node n that have the classification c(x).
  - Assign a probability to each of the possible values of A.

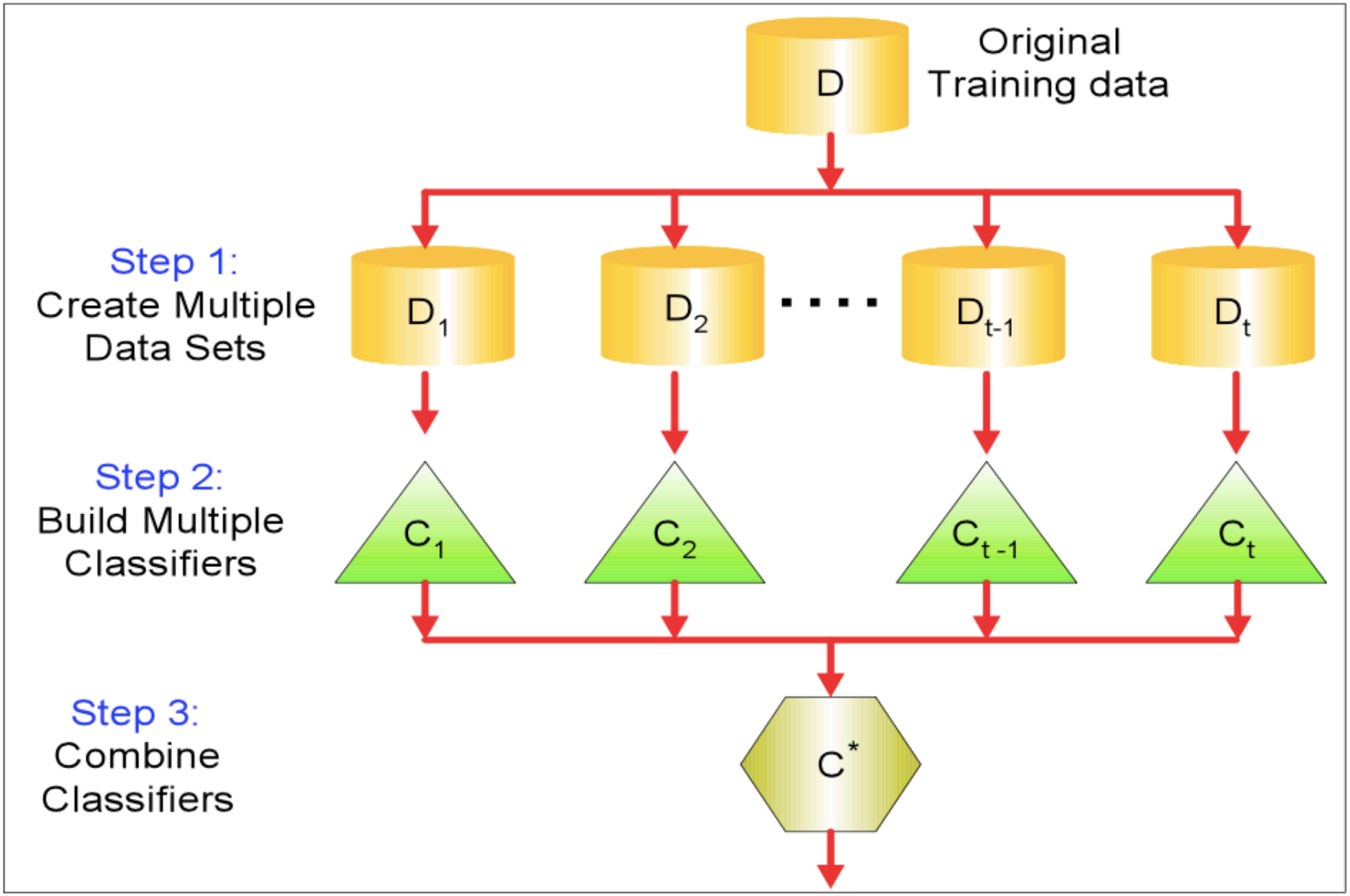
Ensemble Methods

Bagging: Bootstrap Aggregation

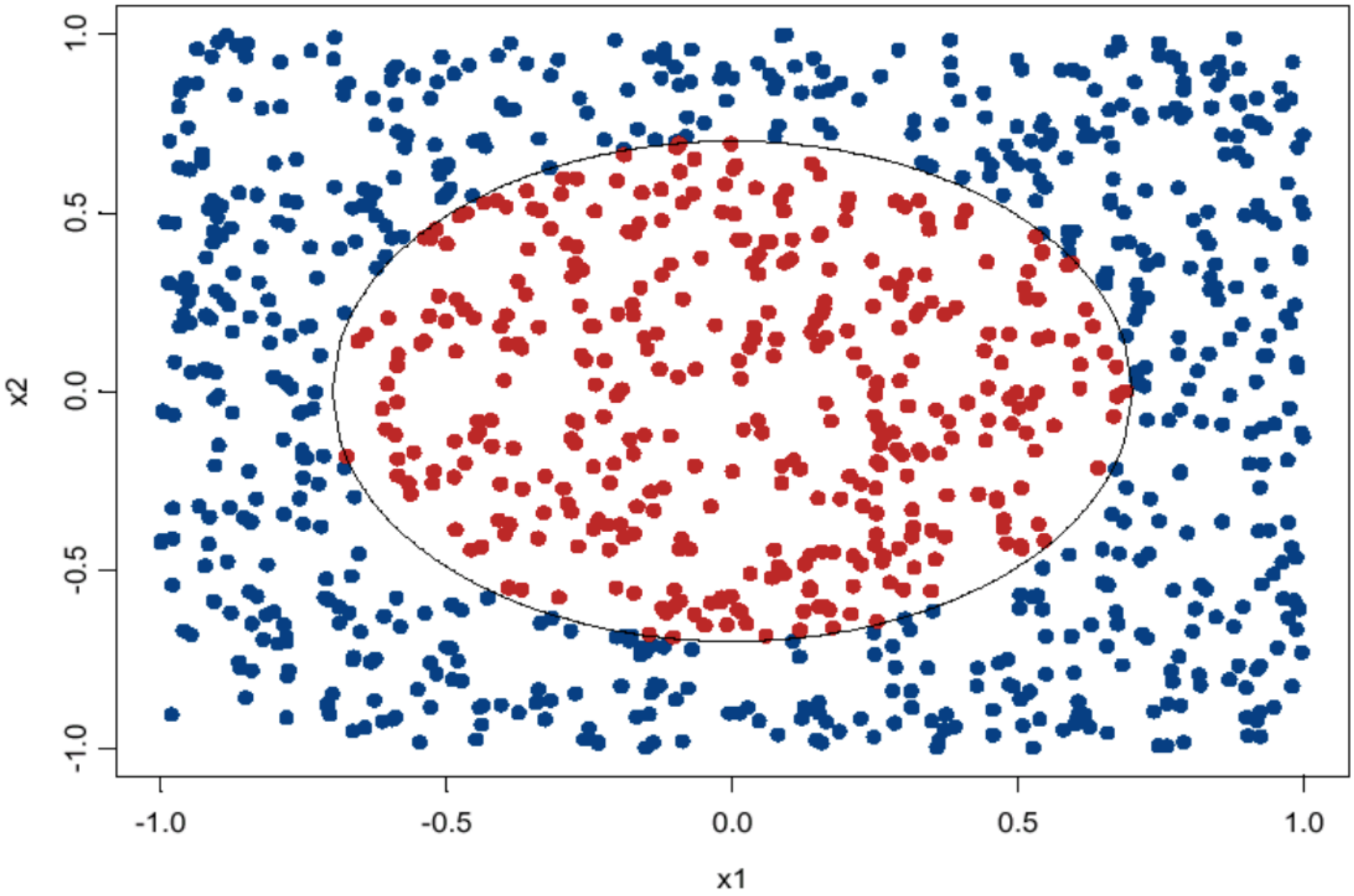
- Leo Breiman (1994)
- Take repeated bootstrap samples from training set *D*
- *Bootstrap sampling*: Given set *D* containing *N* training examples, create *D'* by drawing *N* examples at random with replacement from *D*.

- Bagging:
  - Create *k* bootstrap samples D_1 … D_k.
  - Train distinct classifier on each D_i.
  - Classify new instance by majority vote / average.

General Idea

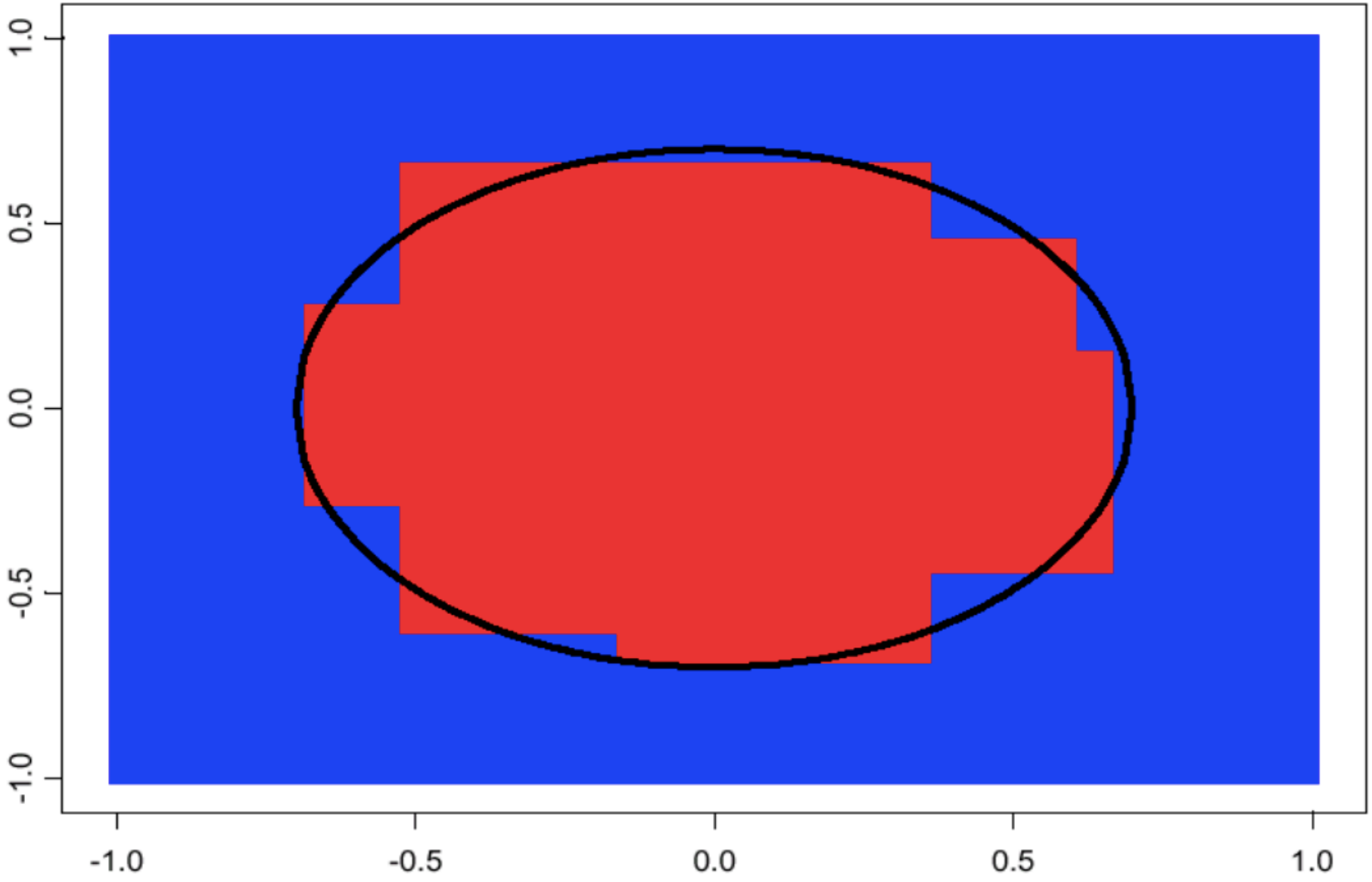


Bagging Example

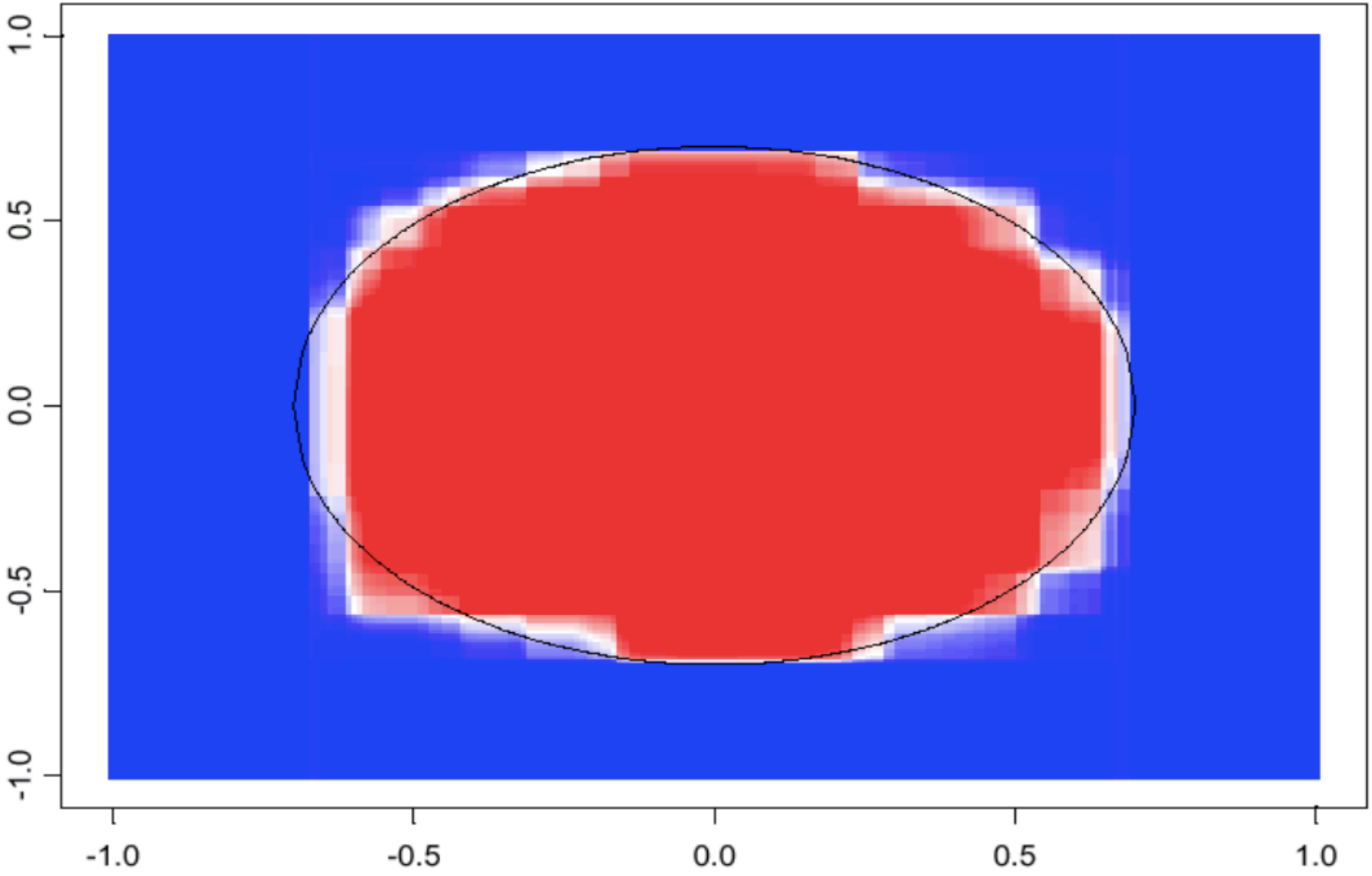


decision tree learning algorithm; very similar to ID3

CART decision boundary



100 bagged trees



shades of blue/red indicate strength of vote for particular classification

Random Forests

- Ensemble method specifically designed for decision tree classifiers

- Introduce two sources of randomness: "Bagging" and "Random input vectors"
  - Bagging method: each tree is grown using a bootstrap sample of training data
  - Random vector method: At each node, best split is chosen from a random sample of *m* attributes instead of all attributes

Random Forests

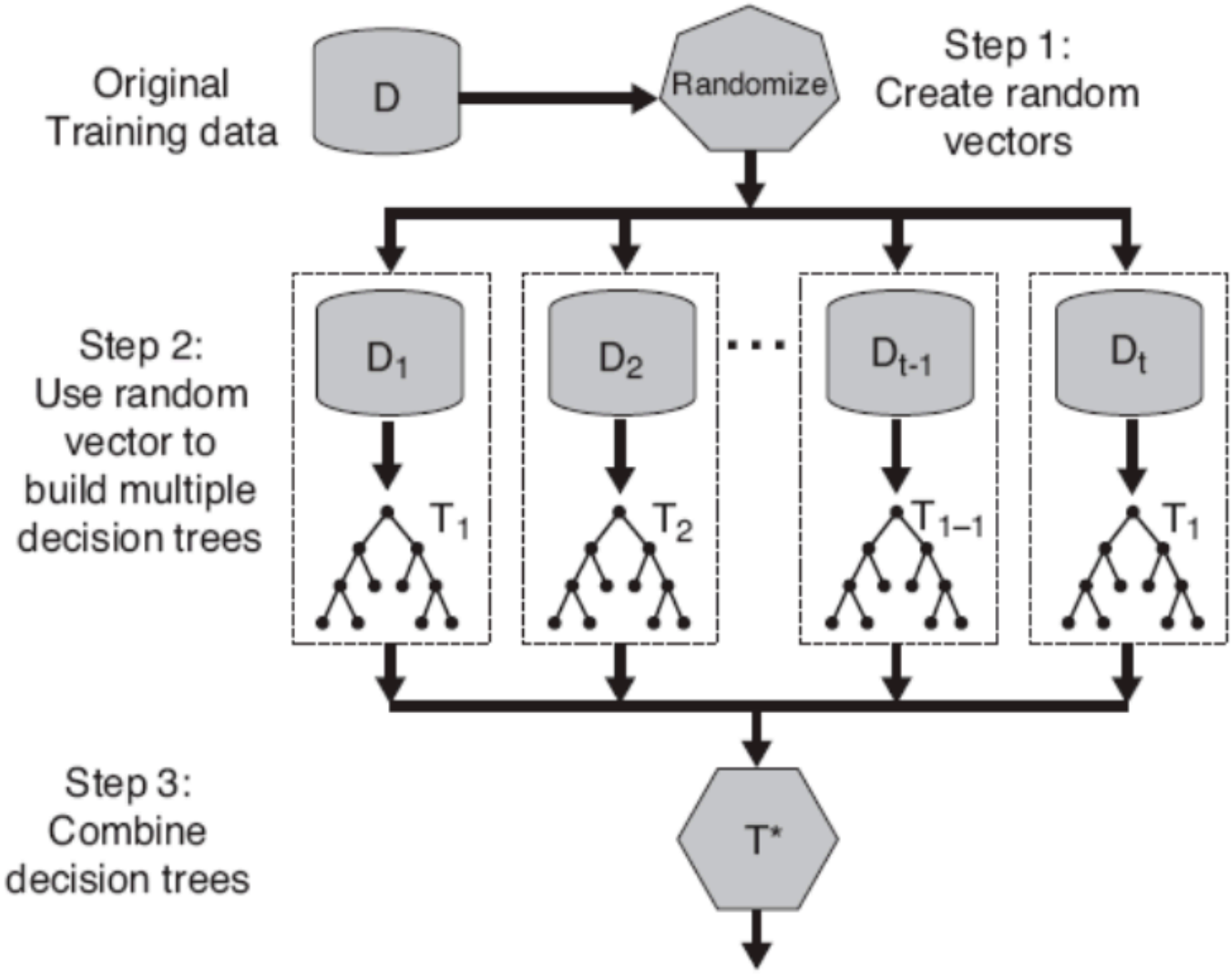


Figure 5.40. Random forests.

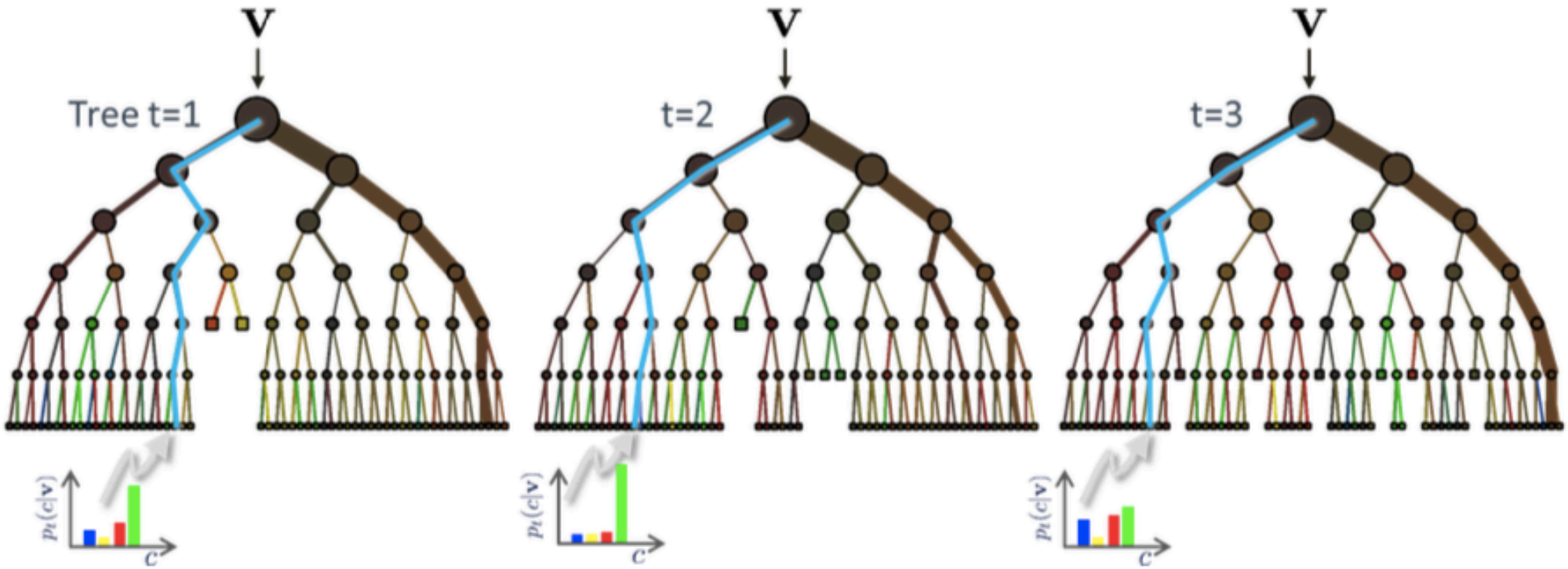


Fig. 3.2: **Classification forest testing.** During testing the same unlabelled test input data $\mathbf{v}$ is pushed through each component tree. At each internal node a test is applied and the data point sent to the appropriate child. The process is repeated until a leaf is reached. At the leaf the stored posterior $p_t(c|\mathbf{v})$ is read off. The forest class posterior $p(c|\mathbf{v})$ is simply the average of all tree posteriors.

Random Forests Algorithm

1. For $b = 1$ to B:

   (a) Draw a bootstrap sample $\mathbf{Z}^*$ of size N from the training data.

   (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.

      i. Select m variables at random from the p variables.

      ii. Pick the best variable/split-point among the m.

      iii. Split the node into two daughter nodes.

2. Output the ensemble of trees $\{T_b\}_1^B$.

To make a prediction at a new point x:

Regression: $\hat{f}_{\text{rf}}^B(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$.

Classification: Let $\hat{C}_b(x)$ be the class prediction of the bth random-forest tree. Then $\hat{C}_{\text{rf}}^B(x) = \textit{majority vote}\ \{\hat{C}_b(x)\}_1^B$.